Cairo University - Faculty Of Engineering
Computer Engineering Department
Computer Graphics - Spring 2025

One Time Pad Stream Cipher

Project Report for Computer Security Course Project

Submitted to

Prof. Samir Shaheen
Eng. Salma Abd El-Moniem

Submitted by Team

Name	Section	BN	Workload
Amir Anwar	1	15	Code & Report
George Magdy	1	18	Code & Report

May 2, 2025

1 Introduction

Project Idea

This project implements a secure communication system using a One-Time Pad (OTP) encryption scheme. The system ensures confidentiality, integrity, and secure key exchange between a sender and a receiver.

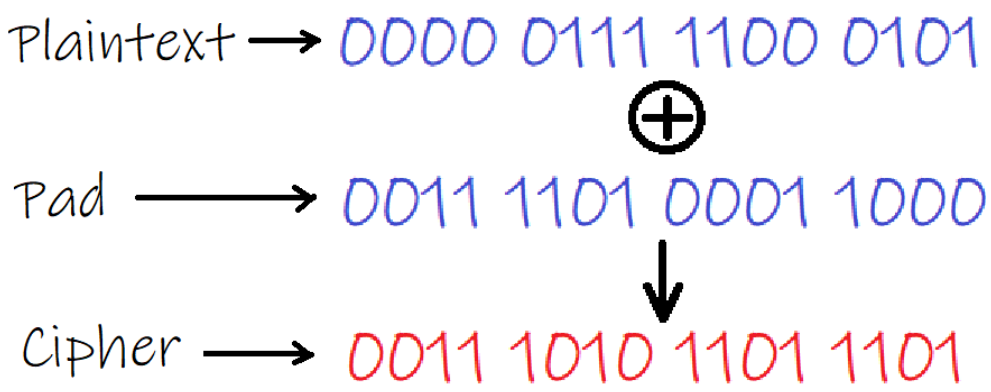

Figure 1: One Time Pad

2 Chosen Algorithms

- **Key Exchange:** We chose The Diffie-Hellman algorithm for its ability to securely exchange keys over an insecure channel
- **Message Encryption:** We implemented our one-time pad with a stream cipher based on LCG (Linear Congruential Generator) for pseudo-random number generator
- **Seed Encryption:** We chose AES with a shared key derived from Diffie-Hellman to Encrypt the seed used in the LCG to ensure secure communication
- **Message Integrity:** We used HMAC (Hash base message authentication code) to verify the authenticity of the transmitted data

3 Functionalities

3.1 Sender

1. Starts a server and waits for a connection.
2. Performs key exchange using Diffie-Hellman.
3. Encrypts the seed and sends it along with the HMAC.
4. Encrypts the input file in chunks and transmits the encrypted data.
5. Sends an end-of-transmission signal.

3.2 Receiver

1. Connects to the sender.
2. Performs key exchange using Diffie-Hellman.
3. Verifies the HMAC and decrypts the seed.
4. Decrypts the received data in chunks and writes it to the output file.

4 Algorithms Examples

4.1 Diffie-Hellman Key Exchange

Alice and Bob - God Bless Alice and Bob :) - need to publicly agree on:

- A prime number: $p = 23$
- A base (generator): $g = 5$

Alice picks a secret number $a = 6$, and calculates:

$$A = g^a \mod p = 5^6 \mod 23 = 8$$

Bob picks his own secret $b = 15$, and calculates:

$$B = g^b \mod p = 5^{15} \mod 23 = 2$$

They exchange A and B.
Now, each one calculates the shared secret:

- Alice computes: $K = B^a \mod p = 2^6 \mod 23 = 18$
- Bob computes: $K = A^b \mod p = 8^{15} \mod 23 = 18$

So, both end up with the same shared key: **18**. Even though an attacker sees A and B, they can't easily compute the shared secret without solving a hard math problem (the discrete logarithm problem).

4.2 Linear Congruential Generator (LCG)

LCG is a simple way to generate a sequence of pseudo-random numbers using the formula:

$$x_{n+1} = (ax_n + c) \mod m$$

Let's use the following small values for demonstration:

- Multiplier $a = 5$
- Increment $c = 1$
- Modulus $m = 16$
- Seed $x_0 = 7$

Now we compute a few numbers in the sequence:

- $x_1 = (5 \cdot 7 + 1) \mod 16 = 36 \mod 16 = 4$
- $x_2 = (5 \cdot 4 + 1) \mod 16 = 21 \mod 16 = 5$
- $x_3 = (5 \cdot 5 + 1) \mod 16 = 26 \mod 16 = 10$

So, the generated sequence starts as: $7, 4, 5, 10, \ldots$

4.3 HMAC (Hash-based Message Authentication Code)

HMAC is used to make sure that a message has not been tampered with and really comes from the sender. It uses a secret key and a hash function (like SHA-256).

HMAC works by applying the hash function twice, like this:

$$\text{HMAC}(key, message) = H((key \oplus \text{opad}) \,\|\, H((key \oplus \text{ipad}) \,\|\, \text{message}))$$

Where:

- H is the hash function (e.g., SHA-256)
- $\oplus$ means XOR
- `opad` and `ipad` are fixed padding values

This result (called the MAC) can be used to verify both the integrity and authenticity of the message.

5 Looking for Optimal Parameters

5.1 LCG Parameters

Based on *Numerical Recipes* (ranqd1, Chapter 7.1 "An Even Quicker Generator", Eq. 7.1.6) with parameters attributed to Knuth and H. W. Lewis [1]. Commonly used values (see Wikipedia) are:

- Multiplier: $a = 1\,664\,525$
- Increment: $c = 1\,013\,904\,223$
- Modulus: $m = 2^{32} = 4\,294\,967\,296$

5.2 Diffie–Hellman Parameters

Using the 2048-bit MODP Group from RFC 3526, Section 3.2 [2]. The prime p is given in hexadecimal, and $g = 2$:

$p =$ `FFFFFFFF FFFFFFFF C90FDAA2 2168C234 C4C6628B 80DC1CD1`
`29024E08 8A67CC74 020BBEA6 3B139B22 514A0879 8E3404DD`
`EF9519B3 CD3A431B 302B0A6D F25F1437 4FE1356D 6D51C245`
`E485B576 625E7EC6 F44C42E9 A637ED6B 0BFF5CB6 F406B7ED`
`EE386BFB 5A899FA5 AE9F2411 7C4B1FE6 49286651 ECE45B3D`
`C2007CB8 A163BF05 98DA4836 1C55D39A 69163FA8 FD24CF5F`
`83655D23 DCA3AD96 1C62F356 208552BB 9ED52907 7096966D`
`670C354E 4ABC9804 F1746C08 CA18217C 32905E46 2E36CE3B`
`E39E772C 180E8603 9B2783A2 EC07A28F B5C55DF0 6F4C52C9`
`DE2BCBF6 95581718 3995497C EA956AE5 15D22618 98FA0510`
`15728E5A 8AACAA68 FFFFFFFF FFFFFFFF`,

$g = 2$.

References

[1] W. H. Press *et al.*, *Numerical Recipes in C: The Art of Scientific Computing*, 2nd ed., Cambridge University Press, 1997.

[2] S. Fluhrer and N. McDonald, *RFC 3526: More Modular Exponential (MODP) Diffie–Hellman groups for Internet Key Exchange (IKE)*, Internet Engineering Task Force, April 2003.